

The Linux `perf` Tool: A Deep-Dive for Database Performance Engineering (PostgreSQL & MySQL/MariaDB)

TL;DR

- `perf` (`perf_events`) is the right first-reach OS-level profiler for production databases because it samples via CPU hardware counters and the kernel scheduler *without stopping the process* — typically 1-5% overhead at 99 Hz — which is the decisive contrast with `gdb`'s all-stop attach that can freeze an entire PostgreSQL cluster or stall `mysqld`. Use it to answer "where are CPU cycles going, and why are backends off-CPU?"
 - The single biggest practical obstacle for both engines is broken stacks: stock PostgreSQL and MySQL/MariaDB packages are built `-O2 -fomit-frame-pointer`, so `perf record -g` (frame-pointer mode) yields `[unknown]` frames. The fixes, in order of preference, are `--call-graph dwarf` (needs `libunwind` + heavy stack copies), `--call-graph lbr` (Intel LBR, shallow, unavailable in most clouds), or rebuilding with `-fno-omit-frame-pointer` and installing `-dbgsym`/`-debuginfo`. PostgreSQL additionally ships USDT/SDT probes (via `--enable-dtrace`, default in PGDG builds) that `perf probe` can arm.
 - On-CPU profiling alone is insufficient for databases, which spend much of their time *blocked* on LWLock/InnoDB-mutex waits, buffer I/O, and WAL/redo flushes. Pair on-CPU flame graphs with **off-CPU analysis** (scheduler tracepoints / eBPF), `perf c2c` for NUMA cache-line contention in shared memory (buffer headers, lock partitions, ProcArray), and the engine's own instrumentation (PostgreSQL wait events / `pg_stat_io`; MySQL `performance_schema`). Note that managed cloud databases (RDS/Aurora, Cloud SQL) do not expose `perf`.
-

Key Findings

1. `perf` is the userspace front-end (in the kernel tree at `tools/perf`) to the kernel `perf_events` subsystem, driven by the `perf_event_open(2)` syscall. It was added to Linux in 2.6.31 (2009), originally "Performance Counters for Linux" (PCL), then renamed `perf_events`. It unifies four event sources behind one interface: PMU hardware counters, software events, static tracepoints, and dynamic kprobes/uprobes. (postgresql)
 2. Two operating modes: *counting* (`perf stat`), aggregates totals via `read(2)` on the counter fd) and *sampling* (`perf record`/`perf top`), driven by counter overflow raising a Performance Monitoring Interrupt (PMI). Interrupt-based sampling suffers **skid** — the recorded instruction pointer is the resumption instruction, not the one that caused the event, because μ ops execute out-of-order. **Precise sampling** (Intel PEBS, AMD IBS), invoked with the `:p`/`:pp`/`:ppp` modifiers, corrects this.
 3. Call-graph collection has three mechanisms with sharp tradeoffs: frame pointers (`--call-graph fp`, needs `-fno-omit-frame-pointer`, cheapest, breaks on optimized DB builds), DWARF (`--call-graph dwarf`, copies the user stack on each sample — large `perf.data`, higher overhead, works without frame pointers via `libunwind`'s `.eh_frame`), and LBR (`--call-graph lbr`, hardware Last Branch Record, limited depth, and *disabled in most cloud environments*).
 4. For PostgreSQL, the canonical workflow is to find a backend via `SELECT pg_backend_pid()` and run `perf record -F 99 -p PID --call-graph dwarf -- sleep N`, or profile system-wide with `perf record -a -g`. PostgreSQL exposes SDT probes such as `sdt_postgresql:query_execute_start`, `checkpoint_start/done`, `buffer_sync_start/done`, and `transaction_start`, and `perf probe` can inject dynamic uprobes on arbitrary functions (e.g., `XLogFlush`, `standard_ExecutorStart`) and even capture arguments (`queryDesc->sourceText:string`).
 5. For MySQL/MariaDB, `perf record -F 99 -g -p $(pgrep -x mysqld)` captures all threads of the thread-per-connection server; use `-t TID`/`--per-thread` to isolate a connection or replication thread. Percona's support team routinely converts these to flame graphs via `perf script | stackcollapse-perf.pl | flamegraph.pl`. InnoDB mutex/rw-lock contention that is *not* instrumented in `performance_schema` (a documented gap — e.g., block rw-locks) is frequently only visible through `perf` sampling or off-CPU analysis.
 6. Access is gated by `perf_event_paranoid` and `kptr_restrict`. Kernel default is 2 (userspace profiling only), which is why unprivileged `perf` shows only userspace or fails on kernel/CPU events. Lower it (or grant `CAP_PERFMON`/`CAP_SYS_ADMIN`) for full profiling. This, plus the requirement for host PMU access, is why `perf` is generally unavailable in managed DBaaS and often in containers. (Sysctl Explorer)
-

Details

1. What `perf` Is and Its Architecture

Origin and structure. `perf_events` was merged into the Linux kernel at **2.6.31** as "Performance Counters for Linux," then generalized and renamed. The userspace `perf` binary lives *in the kernel source tree* under `tools/perf` and is developed in lockstep with the kernel; it is GPLv2. The kernel side comprises architecture-specific PMU drivers that program the hardware, manage counters (enable/disable/read/reset), handle overflow interrupts for sampling, and transfer results to userspace via ring buffers `mmap`'d into the tool's address space.

The syscall boundary. Everything flows through `perf_event_open(2)`, which takes a `struct perf_event_attr` (the `type` field selects `PERF_TYPE_HARDWARE`, `PERF_TYPE_SOFTWARE`, `PERF_TYPE_HW_CACHE`, tracepoint, etc.; `config` selects the specific event) plus `pid` and `cpu` arguments. The pid/cpu matrix defines scope: `pid>0, cpu=-1` measures a process on any CPU; `pid=-1, cpu>=0` measures all processes on one CPU (system-wide per-CPU, requiring `CAP_PERFMON`/`CAP_SYS_ADMIN` or `perf_event_paranoid < 1`). The returned fd is used with `read(2)` for counting and `mmap(2)` for sampling. A single counter read costs roughly **1.5–3.0 µs** on typical x86 (measured in the PerfWeb study), which bounds how fine-grained polling can be.

Event taxonomy (unified interface).

- **Hardware events (PMCs):** cycles, instructions, cache-references/misses, branch/branch-misses, bus-cycles — programmed into the CPU's Performance Monitoring Unit. PMUs have a small number of counters (a mix of *fixed-function* and *programmable* / model-specific), which forces multiplexing when you request more events than counters exist.
- **Software events:** `cpu-clock`, `task-clock`, `context-switches` (`cs`), `cpu-migrations`, `page-faults` (minor/major) — kernel-maintained counters that need no PMU.
- **Static tracepoints:** compiled-in kernel instrumentation (`sched:sched_switch`, `block:block_rq_issue`, `syscalls:sys_enter_fsync`, `ext4:*`, `vmscan:*`).
- **Dynamic tracing:** kprobes (kernel) and uprobes (userspace) placed at runtime via `perf probe`, with no recompilation.

Sampling mechanics and skid. Sampling can be frequency-driven (`-F 99`) → ~99 samples/sec, kernel adjusts the period) or period-driven (`-c N`) → one sample every N events). Overflow raises a PMI; the handler captures IP, registers, and optionally the call stack. Because the pipeline is out-of-order, the captured IP skids past the culprit instruction. Brendan Gregg notes this matters for *event* profiling (e.g., attributing an LLC miss to the wrong line) far more than for *timed* profiling. PEBS (Intel) and IBS (AMD) record the precise architectural state at the event; request them with modifiers — e.g., `perf record -e cycles:up` for precise user-space cycles.

Symbol resolution. Kernel symbols come from `/proc/kallsyms` (subject to `kptr_restrict`); userspace symbols from ELF symbol tables and separate debuginfo, matched via **build-id**. JIT/interpreted code (Java, Node) needs a `/tmp/perf-PID.map` file; this is not relevant to C-compiled Postgres/MySQL but is the same machinery that makes `perf buildid-cache --add` and `perf archive` work for offline analysis.

2. Repository, Versioning, and Packaging

`perf` ships with the mainline kernel (`git.kernel.org`, `tools/perf`) and is versioned with it. On Debian/Ubuntu it is `linux-tools-common` + `linux-tools-$(uname -r)` (the PostgreSQL wiki and Percona both stress that **the perf binary must match the running kernel**); on RHEL/Fedora it is the `perf` package. Newer perf tools generally run against older kernels and vice-versa, with per-feature caveats (some subcommands like `perf c2c` require Linux 4.10+; system-wide `-g` without `-a` requires ≥4.11). Documentation: the man pages (`man perf`, `man perf-record`, `man perf-stat`, `man perf_event_open`), the perf wiki at `perf.wiki.kernel.org`, Vince Weaver's unofficial perf-events page, and Brendan Gregg's `perf.html` — the most comprehensive practitioner reference.

3. Core Workflow and Subcommands

`perf stat` — counting mode. Default set: task-clock, context-switches, cpu-migrations, page-faults, cycles, instructions (with derived IPC), branches, branch-misses, cache-references, cache-misses. Key flags: `-e` (event selection, wildcards like `'syscalls:sys_enter_*'`), `-a` (system-wide), `-p PID`, `-d`/`-dd` (detailed, adds L1/LLC/TLB), `-r N` (repeat and average), `-I 1000` (interval printing per ms), plus per-socket/per-core aggregation. Raw PMCs can be given as `-e r003c` or `-e cpu/event=0x0e,umask=0x01,inv,cmask=0x01/`. (brendangregg)

A database-relevant one-liner from the PostgreSQL wiki:

```
sudo perf stat -e block:block_rq_*,syscalls:sys_enter_write,syscalls:sys_enter_fsync -a -r 5 -- \
psql -q -U postgres postgres -c "create table x as select a from generate_series(1,1000000) a;"
```

`perf record` — sampling to `perf.data`. `-F` frequency, `-c` period, `-g`/`--call-graph {fp,dwarf,lbr}`, `-a`, `-p PID`, `-t TID`, `-e`, `perf report` — TUI or `--stdio`, columns **Overhead % / Command / Shared Object / Symbol**, with `--sort`, `-n` (sample counts), `-g folded`, and `--children` (cumulative) vs self overhead. `perf annotate` — per-instruction source+assembly cost attribution. `perf top` — live system-

wide (`perf top -a, -ns comm, dso`). `perf list` — enumerate events. `perf script` — dump raw samples (the flame-graph input path). `perf trace` — lower-overhead strace alternative.

Specialized subcommands (highly database-relevant): `perf probe` (dynamic kprobes/uprobes), `perf c2c` (cache-to-cache / false-sharing, Linux 4.10+), `perf mem` (load/store latency profiling), `perf sched` (scheduler latency), `perf lock` (lock contention), `perf kmem`, `perf kvm`, `perf ftrace`, `perf bench`, `perf test`, `perf diff` (compare two `perf.data` — ideal for before/after a config or version change), and `perf archive` (bundle `perf.data` with symbols for offline `perf report`).

4. Advanced Analysis Techniques

Flame graphs. Brendan Gregg's FlameGraph toolkit turns folded stacks into interactive SVGs:

```
perf record -F 99 -a -g -- sleep 30
perf script | ./stackcollapse-perf.pl | ./flamegraph.pl > out.svg
```

Width = proportion of samples (time on CPU); the y-axis is stack depth (caller→callee). Iccle graphs invert it; **differential flame graphs** color a diff between two profiles. This visualizes the PostgreSQL executor's Volcano/pull model and MySQL's parser→optimizer→handler→InnoDB path directly. (Percona)

Off-CPU analysis — essential for databases. On-CPU sampling misses time spent blocked (I/O, locks, sleeping). Off-CPU analysis measures *off-CPU time with stacks* by tracing the scheduler's switch-out path (`sched:sched_switch`) or, more efficiently, eBPF (`offcputime`) from bcc, or bpftrace). Gregg's own benchmark on an 8-CPU Linux 4.15 host under **heavy MySQL load at 102k context-switches/sec** quantifies the cost difference: tracing every scheduler event with `perf` caused a **9% throughput drop** (occasionally 12% during flushes), producing a **224 MB file for 10 seconds** of tracing, versus **6% with eBPF** in-kernel aggregation — the reason eBPF is preferred for off-CPU work in production. Combine on-CPU + off-CPU for a complete "100% of thread time" picture; this is exactly how you attribute a slow database transaction between compute, lock waits, and I/O.

Top-down Microarchitecture Analysis (TMA). `perf stat --topdown` (and Andi Kleen's `pmu-tools`/`toplev`) decomposes pipeline slots into four categories — **Retiring, Bad Speculation, Frontend-Bound, Backend-Bound** — then drills into sub-levels (e.g., Backend → Memory-Bound → L3/DRAM). For a CPU-bound OLTP database this quickly tells you whether you are limited by instruction supply (frontend/i-cache/iTLB — relevant to huge pages for the text segment), memory stalls (backend/LLC/DRAM latency), or branch mispredicts. *(This subsection is drawn from perf/pmu-tools documentation rather than a database-specific case study — treat the specific per-workload breakdown as needing local measurement.)*

Hardware deep dives. Cache-miss profiling: `perf record -e LLC-load-misses -c 100 -ag`; TLB: `perf stat -e dTLB-load-misses, iTLB-load-misses` (iTLB pressure is a classic argument for huge pages on large-text-segment binaries and for `huge_pages` on Postgres shared buffers). Stalled-cycles-frontend/backend expose pipeline bottlenecks.

NUMA and `perf c2c`. `perf c2c record -a -- sleep 10; perf c2c report` uses memory-access sampling (PEBS/IBS load-latency) to find **hot cache lines shared across cores/NUMA nodes**, distinguishing true from false sharing and reporting **HITM** (hit-in-modified — a remote core supplying a modified line, the signature of contention). On multi-socket PostgreSQL this is the tool for cache-line contention in buffer headers, lock-manager partitions, and the ProcArray; on MySQL it finds contended InnoDB structures. It reports the offending symbol, cache-line offset, and node locality.

Multiplexing. When requested events exceed available counters, the kernel time-multiplexes and scales the reported count, printing the `enabled/running` fraction (the `[%]` column). Scaled counts are estimates — for accuracy, keep the event set within the hardware counter budget or accept the documented approximation.

5. PostgreSQL-Specific Profiling

Targeting. System-wide (`perf record -a -g`) captures the whole cluster; per-backend targeting is the more surgical approach:

```
-- in psql session T1:
SELECT pg_backend_pid(); -- e.g. 12942
```

```
# shell T2:
perf record -F 99 -p 12942 --call-graph dwarf -- sleep 60
# then run the workload in T1
```

You can also profile all Postgres processes with `-u postgres`. EDB/2ndQuadrant emphasize `perf`'s **non-intrusiveness**: no debugger attach, no restart, no recompile needed for kernel-side data — you can test hypotheses on a live system with minimal impact.

The frame-pointer problem. Stock/PGDG PostgreSQL is `(-O2 -fomit-frame-pointer)` (the x86-64 default), so `(perf record -g)` produces stacks that are almost entirely `([unknown])`. Three remedies:

1. `(--call-graph dwarf)` on a perf built with libunwind (works on stock binaries; heavier, larger files);
2. LBR where available (rare in cloud);
3. Rebuild: `(./configure CFLAGS="-fno-omit-frame-pointer -ggdb" --enable-debug)` (keep `(-O2)`, add frame pointers) and install `(postgresql-*-dbgsym)(-debuginfo)`. The industry has shifted toward shipping frame pointers by default (Fedora 38+ and Ubuntu 24.04 enable them), which increasingly makes `(perf record -g)` work out-of-the-box on newer distro Postgres packages.

Built-in USDT/SDT probes. With `(-enable-dtrace)` (requires SystemTap at build time; **enabled by default in PGDG binaries**), PostgreSQL compiles static probes visible via `(perf list | grep sdt)`. Examples include `(sdt_postgresql:query__execute__start)`, `(checkpoint__start)`/`(checkpoint__done)`, `(buffer__sync__start)`/`(buffer__sync__done)`, `(transaction__start)`. They must be armed first:

```
perf probe sdt_postgresql:query__execute__start
perf record -e sdt_postgresql:query__execute__start_1 -aR sleep 1
```

New probes are added by editing `(src/backend/utils/probes.d)` (lowercase, doubled underscores) and calling `(TRACE_POSTGRES_*_*)` macros in the source (e.g., `(TRACE_POSTGRES_TRANSACTION_START)` in `(StartTransaction)` in `(xact.c)`).

Dynamic probes and argument capture (no patching, no gdb):

```
perf probe -x $(which postgres) XLogFlush
perf probe -x $(which postgres) standard_ExecutorStart 'queryDesc->operation' 'queryDesc->sourceText:string'
perf record -e probe_postgres:standard_ExecutorStart -u postgres -o - | perf script -i -
```

This lets you watch WAL activity `(XLogFileInit)`, `(XLogFileOpen)`, `(XLogFlush)` or every planned query live. Capturing arguments substantially raises overhead and trace size — use sparingly.

Concrete diagnostic workflows.

- *CPU-bound slow query*: flame graph reveals hot paths such as `(heap_hot_search_buffer)`, `(ExecInterpExpr)`, `(slot_deform_heap_tuple)`, hashing `(ExecHashTableInsert)`, or `(tuplesort)`. Robert Haas (EDB) documented using `(perf record -g -e cs)` (context-switches) to trace LWLock contention, which led directly to a patch removing redundant CLOG access from `(heap_hot_search_buffer)` — a real committed optimization found *via perf*.
- *Lock-bound*: `(perf record -e context-switches -ag)` or off-CPU stacks pointing at `(LWLockAcquire)`.
- *I/O-bound*: off-CPU time in buffer reads / `(smgrread)`, plus `(perf stat -e block:*)` and correlation with `(pg_stat_io)`.
- *Autovacuum / checkpoint / WAL*: target the autovacuum worker PID or checkpoint; combine the `(checkpoint__start/done)` and `(buffer__sync__*)` SDTs with `(block:block_rq_issue)` and `(syscalls:sys_enter_fsync)`.
- Haas also cautions that context-switch profiling is *frequency-biased*, not wall-clock: it over-weights code that switches out often-but-briefly and under-weights rare-but-long blocking. This is precisely the gap that eBPF off-CPU *time* profiling fills.

Correlation with in-server instrumentation. Use `(EXPLAIN (ANALYZE, BUFFERS))`, `(pg_stat_statements)`, wait events in `(pg_stat_activity)` (and aggregated wait-event views), and `(pg_stat_io)` to localize the problem, then use `(perf)` to explain *why* a hot function or wait is expensive at the instruction and cache-line level. Andres Freund's Citus Con 2022 talk "Analyzing Postgres performance problems using perf and eBPF" is the canonical practitioner walkthrough of this combined methodology.

6. MySQL / MariaDB-Specific Profiling

Threading and attribution. `(mysqld)`/`(mariadb)` is thread-per-connection; `(perf record -p PID)` captures all threads, `(-t TID)` isolates one (a specific connection or a replication applier), and `(--per-thread)` breaks counts out per thread. Percona's standard capture:

```
sudo perf record -a -F 99 -g -p $(pgrep -x mysqld) -- sleep 10
sudo perf script | ~/src/FlameGraph/stackcollapse-perf.pl | ~/src/FlameGraph/flamegraph.pl > flame.svg
```

Symbols/frame pointers. Same story as Postgres: install `(linux-tools-$(uname -r))`, install the matching `(-dbgsym)`/`(-debuginfo)`, and either use `(--call-graph dwarf)` or build with `(-fno-omit-frame-pointer)` to resolve `([unknown])` frames into parser/optimizer/handler/InnoDB functions.

InnoDB internals. Flame graphs and sampling expose hot paths through the SQL layer, optimizer, the handler interface, and InnoDB (buffer-pool operations, page flushing, redo log, adaptive hash index). Crucially, **some InnoDB rw-lock/mutex contention is invisible in**

`performance_schema` — Percona documents MySQL bug #74280, a covering-index regression where "the block rw-lock isn't instrumented," so the contention was only diagnosable with `perf`. This is the central argument for reaching past in-server instrumentation to OS-level `perf` + off-CPU analysis for mutex waits.

`performance_schema` vs `perf`. `performance_schema` gives you *instrumented*, MySQL-aware wait/statement/stage/memory data with low, tunable overhead and no root; `perf` gives you *everything the CPU and kernel see* (including uninstrumented locks, libc/allocator, kernel I/O) but needs privileges and symbol setup. Use P_S to localize, `perf` to explain. Percona also composes flame graphs from `pt-pmp` (poor-man's profiler) output, but warns `pt-pmp`/`pt-stalk` use **`gdb` under the hood and will stall MySQL** — the very intrusiveness `perf` avoids.

Dynamic probes / USDT. MySQL historically shipped DTrace providers; on Linux you can inject uprobes on `mysql` functions with `perf probe -x $(which mysql) <symbol>` (requires non-static, visible symbols or a `-ggdb` build).

Concrete workflows. OLTP CPU hotspots (parsing/optimization overhead, `row_search_mvcc`, B-tree search), InnoDB spinlock/mutex contention (on-CPU spin paths + off-CPU mutex waits), memory-allocation hotspots (profile `malloc`/jemalloc/tcmalloc paths — often a large flame-graph slice under high connection churn), and replication-thread profiling via `-t TID`. Percona (very active here), the "Flame Graphs MySQL" Percona Live sessions, and Brendan Gregg's MySQL flame-graph writeups are the reference case studies. `Percona`

7. Practical Considerations and Best Practices

Overhead and non-intrusiveness — the `gdb` contrast. `perf` sampling at 99 Hz is typically low single-digit-percent overhead and does **not** stop the target — the defining difference from `gdb`, whose all-stop attach freezes the process (and, for a hot primary, can cascade into a cluster-wide stall). Gregg's MySQL benchmark above quantifies the *heavier* end (full scheduler tracing): 9% with `perf` event-dumping vs 6% with eBPF summarization. For routine CPU profiling, overhead is far lower.

Frequency choice. Use **99 Hz** (not 100) deliberately to avoid lock-step sampling with periodic kernel/application activity that would bias results; step up to `-F 999` for finer resolution at higher cost. DWARF call graphs are the dominant cost/size driver (each sample copies the user stack — files can reach hundreds of MB); frame pointers are far cheaper.

Privileges. `/proc/sys/kernel/perf_event_paranoid` (kernel default **2**): `-1` = almost all events for all users; `0` = disallow raw/ftrace tracepoints for unprivileged users (kernel+user profiling still allowed); `1` = also disallow CPU-event access; `2` = also disallow kernel profiling (userspace only — the default, which is why unprivileged `perf` often shows only userspace or errors on kernel events). `CAP_PERFMON` (Linux 5.8+) or `CAP_SYS_ADMIN` bypass these. `kptr_restrict` controls kernel-symbol visibility in `/proc/kallsyms`; set to `0` for readable kernel symbols. The `context_switch` attr (Linux 4.3+) gives full switch info even under strict paranoid settings.

Containers/Kubernetes/cloud. `perf` needs host PMU access and elevated capabilities, usually absent in unprivileged containers. Managed databases — **Amazon RDS/Aurora**, **Google Cloud SQL** — do not expose `perf` at all (no shell/PMU). Virtualization may not expose a vPMU to guests; LBR in particular is commonly disabled in clouds (`perf record --call-graph lbr` fails with "PMU Hardware doesn't support sampling/overflow-interrupts"), though some clouds now offer a vPMU.

Reading results correctly. Distinguish **self/overhead** (time *in* the function) from `--children`/**cumulative** (function + callees). Missing symbols show as `[unknown]`; broken/short stacks signal absent frame pointers — fix before drawing conclusions. Sampling is statistical: ensure enough samples (watch the "N samples" line) for confidence, and remember the measure-perturbation tradeoff.

Symbol troubleshooting / portability. Install `-dbgsym`/`-debuginfo`; use `perf buildid-cache --add <binary>` (also how USDT probes get registered); move data between machines with `perf archive` → untar into `~/debug` on the analysis host.

8. Related Tools and Ecosystem

`perf` sits in the middle of the Linux tracing/profiling landscape: **ftrace** (function tracing, some features only reachable via ftrace, wrapped by Gregg's `perf-tools`), **eBPF/bcc/bpftrace** (programmable in-kernel aggregation; `perf` increasingly integrates with BPF, and eBPF is now the preferred engine for off-CPU and high-frequency tracing), **SystemTap** (which PostgreSQL uses to generate its SDT probes), **LTTng**, and historically **DTrace**. Complementary toolkits: Brendan Gregg's **FlameGraph** and **bcc/BPF tools** (`offcputime`, `profile`, `funclatency`); Andi Kleen's **pmu-tools/toplev** for TMA. GUIs and importers for `perf.data`: **Hotspot** (KDAB), the **Firefox Profiler**, **Speedscope**, and continuous-profiling systems like **Grafana Pyroscope**/Polar Signals' Parca (the latter pioneered frame-pointer-free DWARF unwinding for exactly these stripped DB binaries). **magic-trace** and other LBR-based tools capture fine-grained control-flow on Intel.

Where `perf` sits vs neighbors: `gdb` = interactive state inspection, all-stop, unsafe on hot DBs; `strace` = per-syscall tracing, high overhead via ptrace; `tcpdump` = packet capture; `bpfttrace` = programmable dynamic tracing, low overhead, best for off-CPU/custom aggregation; `perf` = **low-overhead statistical CPU/PMU profiler and event tracer, the default first reach for "where are cycles/waits going" on a live database.**

Recommendations

Stage 0 — Prepare (once). Install `(linux-tools-$(uname -r))` (or the `(perf)` RPM) matching the kernel; install `(postgresql-* -dbgsym)` / `MySQL (-debuginfo)`. Set `(perf_event_paranoid)` appropriately (`(1)` for CPU-event access, `(0)` or `(-1)` in trusted, non-shared hosts for full kernel profiling) and `(kptr_restrict=0)`. Verify stacks resolve on a throwaway capture before trusting any profile.

- *Threshold to change:* if `(perf record -g)` shows mostly `([unknown])`, switch to `(--call-graph dwarf)`; if DWARF files are unmanageably large or overhead is visible, rebuild the engine with `(-fno-omit-frame-pointer)` (keep `(-O2)`).

Stage 1 — Triage with counting. `(perf stat -a -I 1000)` and `(perf stat -d -p PID)` to classify the workload: high IPC + high cycles = CPU-bound; high context-switches = lock/scheduler-bound; high `(block:*)/(syscalls:sys_enter_fsync)` = I/O/WAL-bound. Cross-check with `(pg_stat_io)/wait events` or `(performance_schema)`.

Stage 2 — On-CPU flame graph. `(perf record -F 99 -p PID --call-graph dwarf -- sleep 30)` → flame graph. Read the widest towers: executor/optimizer hotspots, allocator, spin paths.

- *Threshold:* if a large fraction of wall-clock time is unaccounted for (backends idle/blocked, not on-CPU), go to Stage 3.

Stage 3 — Off-CPU analysis. Use eBPF `(offcputime)/bpfttrace` (preferred) or `(perf record -e sched:sched_switch -a -g)` for short windows to attribute blocked time to `(LWLockAcquire)`/InnoDB mutex, buffer I/O, or WAL/redo flush. Start at 0.1s trace windows and ratchet up while watching throughput.

Stage 4 — Microarchitecture & NUMA (if CPU-bound and scaling poorly). Run `(perf stat --topdown)/(toplev)` to find frontend vs backend vs bad-speculation limits; run `(perf c2c record/report)` on multi-socket hosts to find HITM/false-sharing on buffer headers, lock partitions, ProcArray, or InnoDB structures. Act on findings via padding/partitioning, NUMA pinning, huge pages (for iTLB/dTLB pressure), or increasing `(innodb_buffer_pool_instances)`.

Stage 5 — Targeted dynamic probes & regression tracking. Use `(perf probe)` (SDTs + uprobes) to instrument specific functions/arguments; use `(perf diff)` to compare profiles across a config change or version upgrade. Always benchmark overhead in staging before running argument-capturing probes or scheduler tracing on production.

Do not run `(pt-pmp)/(pt-stalk)` (gdb-based) or attach `(gdb)` to a hot production primary — prefer `(perf)/eBPF`. **Do not attempt** `(perf)` on managed DBaaS (RDS/Aurora/Cloud SQL); reproduce on a self-managed instance.

Caveats

- **Version/feature drift:** exact `(perf)` behavior, subcommand availability (`(c2c)` ≥4.10, system-wide `(-g)` semantics ≥4.11), and USDT-enablement steps vary by kernel and perf version; verify on your target.
- **Skid and scaled counts:** without PEBS/IBS, event-attributed IPs skid; multiplexed counts are scaled estimates (watch the `(enabled/running)` fraction). Timed CPU profiling is robust to skid; event profiling is not.
- **The Top-down/TMA subsection** reflects perf/pmu-tools documentation and general methodology rather than a published PostgreSQL/MySQL-specific TMA case study; the per-workload category breakdown must be measured locally.
- **Overhead figures** (1–5% for sampling; 6–9%+ for full scheduler tracing) come from Brendan Gregg's benchmarks and community reports on specific hardware/kernels (notably an 8-CPU Linux 4.15 MySQL test); your overhead depends on frequency, call-graph mode, event rate, and context-switch rate.
- **Off-CPU frequency bias:** context-switch-based profiling (`(-e cs)`) over-weights frequent-but-brief blocking and under-weights rare-but-long waits (Robert Haas's caution); use eBPF off-CPU *time* profiling for wall-clock-accurate attribution.
- **`(performance_schema)` gaps:** not all InnoDB locks are instrumented (e.g., block rw-locks per Percona/MySQL bug #74280), so absence of contention in P_S does not prove its absence — confirm with `(perf)`.
- **Cloud/vPMU:** LBR and some PMU features are frequently disabled under virtualization; managed databases expose neither `(perf)` nor the host PMU.